

User Document – ADT Frontlist

Course: CSCI 301 – Computer Science II

Semester: Fall 2025

Student: Dinesh Seveti

Project: Assignment 7 – ADT FrontList

Objective

The goal of this project is to implement an Abstract Data Type (ADT) called **FrontList**, which restricts all operations to the **front (first element)** of a linked list. This project demonstrates three different object-oriented relationships in C++:

Version	Relationship	Description
FrontListHasA	<i>Composition (Has-A)</i>	Contains a LinkedList object as a data member.
FrontListIsA	<i>Public Inheritance (Is-A)</i>	Inherits from LinkedList publicly, exposing only safe front operations.
FrontListAsA	<i>Private Inheritance (As-A)</i>	Privately inherits from LinkedList to reuse code without public access.

Files Included

File	Purpose
LinkedList.h / LinkedList.tpp	Generic linked-list template implementation.
FrontListInterface.h	Abstract base interface for the ADT.
FrontListHasA.h	FrontList implemented using composition.
FrontListIsA.h	FrontList implemented using public inheritance.
FrontListAsA.h	FrontList implemented using private inheritance.
FrontListHasATester.cpp	Tester for Has-A version.
FrontListIsATester.cpp	Tester for Is-A version.
FrontListAsATester.cpp	Tester for As-A version.
script.txt	Recorded test runs and outputs.

How to Compile and Run

Manual Compilation

```
g++ -std=c++17 FrontListHasATester.cpp -o hasa
```

```
g++ -std=c++17 FrontListIsATester.cpp -o isa
```

```
g++ -std=c++17 FrontListAsATester.cpp -o asa
```

Run Programs

./hasa

./isa

./asa

Expected Outputs

=== Testing FrontListHasA ===

Front: 30

Front after remove: 20

Empty? Yes

=== Testing FrontListIsA ===

Front: Python

Front after remove: Java

Empty? Yes

=== Testing FrontListAsA ===

Front: C

Front after remove: B

Empty? Yes

Summary of Results

All three implementations correctly:

- Insert items only at the front.
- Remove and retrieve from the front.
- Detect when the list becomes empty.
- Clear all entries without accessing internal positions.

Each tester confirms identical logical behavior through independent implementation strategies.

5. Conclusion

This assignment successfully demonstrates **composition**, **public inheritance**, and **private inheritance** design approaches for implementing an identical ADT behavior. The testing confirms full correctness and reusability of the LinkedList base structure while respecting ADT FrontList restrictions.